
Supervise Thyself: Examining Self-Supervised Representations in Interactive Environments

Evan Racah¹ Chris Pal^{1 2 3}

Abstract

Self-supervised methods, wherein an agent learns representations solely by observing the results of its actions, become crucial in environments which do not provide a dense reward signal or have labels. In most cases, such methods are used for pretraining or auxiliary tasks for "downstream" tasks, such as control, exploration, or imitation learning. However, it is not clear which method's representations best capture meaningful features of the environment, and which are best suited for which types of environments. We present a small-scale study of self-supervised methods on two visual environments: Flappy Bird and *Sonic The Hedgehog*TM. In particular, we quantitatively evaluate the representations learned from these tasks in two contexts: a) the extent to which the representations capture true state information of the agent and b) how generalizable these representations are to novel situations, like new levels and textures. Lastly, we evaluate these self-supervised features by visualizing which parts of the environment they focus on. Our results show that the utility of the representations is highly dependent on the visuals and dynamics of the environment.

1. Introduction

Self-Supervised methods have emerged as powerful methods for pretraining to learn more general representations for complicated downstream tasks in vision (Misra et al., 2016; Fernando et al., 2015; 2017; Wei et al., 2018; Vondrick et al., 2018; Jayaraman & Grauman, 2015; Agrawal et al., 2015; Pathak et al., 2017b; Wang & Gupta, 2015) and NLP (Peters et al., 2018; Subramanian et al., 2018; Mikolov et al., 2013; Conneau & Kiela, 2018). In interactive environments, they have begun to receive more attention due

their ability to learn general features of important parts of the environment without any extrinsic reward or labels (LeCun, 2018). Specifically, self-supervised methods have been used as auxiliary tasks to help shape the features or add signal to sparse reward problems (Mirowski et al., 2016; Jaderberg et al., 2016; Shelhamer et al., 2016). They also have been used in unsupervised pretraining for control problems, (Ha & Schmidhuber, 2018). Moreover, they have been used in imitation learning to push expert demonstrations and agent observations into a shared feature space (Aytar et al., 2018; Sermanet et al., 2017). Lastly, they have been used in intrinsic reward exploration to learn a representation well-suited for doing surprisal-based prediction in feature space (Pathak et al., 2017a; Burda et al., 2018). In each of these cases, the desired feature space learned with self-supervised methods should capture the agent, objects, and other features of interest, be easy to predict, and generalize to unseen views or appearances. However, existing evaluations of these methods do not really shed light on whether the representations learned by these self-supervised methods really robustly capture these things. Instead, these evaluations only evaluate the utility of these methods on the particular downstream task under study. While these types of tasks have been studied theoretically (Hyvarinen et al., 2018; Arora et al., 2019), they have not really been examined empirically in depth. As such, in this paper we examine a few self-supervised tasks where we specifically try to characterize the extent to which the learned features capture the state of the agent and important objects. Specifically, we measure how well the features: capture the agent and object positions and generalize to unseen environments, and lastly, we qualitatively measure what each self-supervised method is focusing on in the environment. We pick Flappy Bird and *Sonic The Hedgehog*TM because they represent simple and complex games respectively in terms of graphics and dynamics. Also, one can change the level and colors of each to make an "unseen" environment to test generalizability.¹

2. The Self-Supervised Methods We Explore

We explore four different approaches for self-supervision in interactive environments: VAE (Kingma & Welling, 2013),

¹Mila, Université de Montréal ²École Polytechnique de Montréal ³Element AI, Montréal, QC, Canada . Correspondence to: Evan Racah <eracah@gmail.com>, Chris Pal <chris.j.pal@gmail.com>.

¹<https://github.com/eracah/supervise-thyself>

temporal distance classification (TDC)(Aytar et al., 2018), tuple verification (Misra et al., 2016), and inverse model (Agrawal et al., 2016; Jayaraman & Grauman, 2015; Pathak et al., 2017a). We also use a randomly initialized CNN as a baseline. All self-supervised models in this study use a *base encoder*, ϕ which is a four layer convolutional neural network, similar in architecture to the encoder used in the VAE in (Ha & Schmidhuber, 2018). The encoder takes as input a single frame in pixel space, x and outputs $f = \phi(x)$, where $f \in \mathbb{R}^{32}$. Depending on the self-supervised task, certain heads, g are placed on top of the encoder, like a deconvolutional decoder or a linear classifier, that take z or multiple concatenated z 's as input. See figure 2 and the appendix for more details.

3. Experiments/Results

3.1. Datasets/Environments

We collect the data for Flappy Bird (Tasfi, 2016) and *Sonic The Hedgehog*TM, the GreenHillZone Act 1 level (Nichol et al., 2018) by deploying a random agent to collect 10,000 frames. At train time we randomly select frames from these 10,000 to form each batch. For the generalization experiments in section 3.3, we use what we call *FlappyBirdNight*, whereby we change the background, the color of the bird and the color of the pipes. For generalization in Sonic, we use *GreenHillZone Act 2*. We describe the dataset collection in more detail in the appendix

3.2. Extracting Position Information

To show whether the self-supervised methods are capable of encoding the position of important objects, we probe the representations learned from these methods with a linear classifier trained to classify the agent position (bucketed to 16). The results of this experiment are displayed in table 1. We find features from the inverse model are the most discriminative for detecting the position of the bird, likely because the inverse model focuses on what parts of the environment it can predictably control. We find features from TDC are the best for localizing the pipe. This is likely the case because TDC focuses on what parts of the environment are most discriminative for separating frames in time and in Flappy Bird, the background and bird stay in one place and the pipes move to the left to simulate the bird moving to the right. Tuple verification features are good for both objects, the pipe and the bird, likely because the position of the pipe relative to the bird is a very important temporal signal, which is discriminative to whether the frames are in order or not. The VAE does not do much better than random features for the small-sized bird, but very respectably for the large pipe, likely due to VAE's preference to capture larger global structure that more contributes to the reconstruction loss.

Sonic, on the other hand, has much more complex dynamics and graphics than Flappy Bird. As a result the classification performance is not as strong. For example, the inverse model does much worse at capturing the position of Sonic. This is likely due to the more inconsistent response of Sonic to action commands. For example, when Sonic is already in the air jumping, the right command has no effect. The ambiguity to which action was called for what pairs of frames, likely causes the inverse model to do bad at its task and thus not learn good features. Moreover, the frame moves up in response to Sonic jumping, so Sonic's exact pixels are not the only thing that change in response to jump, making it tougher for the inverse model to focus in on Sonic. Moreover, sequence verification methods like TDC and tuple verification are also tripped up by Sonic. This is most likely because even though Sonic moves to the right fairly consistently, the background moves in the x position not Sonic. Normally, that would be no problem for TDC and tuple verification, like in Flappy Bird. However, there is no consistent landmark in the background for these methods to use like the pipes in Flappy Bird. VAEs also do worse than they do at Flappy Bird. However, they do relatively better than any other self-supervised methods. Likely, this is because they are not affected by weird dynamics of the environment.

3.3. Generalizing to New Levels and Colors

We can also show how well these features generalize to new situations. Theoretically, if the features are truly, robustly capturing objects of interest, changing the level or the colors of the environment, should not affect a linear classifier's ability to localize objects of interest. We test this out by looking at zero-shot linear probe accuracy with the background, pipe, and bird colors changed for Flappy Bird and on a new level for Sonic. We see these results in table 2. Unsurprisingly, we find the performance decreases for all self-supervised methods in Flappy Bird. Surprisingly, the features from TDC generalize better than the inverse model for classifying bird's location. Potentially, this is because the color of the bird changes and the features from the inverse model are more specific to the exact appearance of the bird from the setup it was trained on. TDC features, on the hand, may encode the bird based on where it is relative to the pipes, and less so on exactly how it looks and for the same reason, TDC features are able to capture the pipes, despite their different color. The VAE features' performance unsurprisingly drops for both objects, as the global structure that they learn to encode completely changes with the new colors in the *FlappyBirdNight* setup.

3.4. Qualitative Inspection of Feature Maps

We show qualitative inspection by superimposing a frames feature map on top of the frame itself. We pick the most

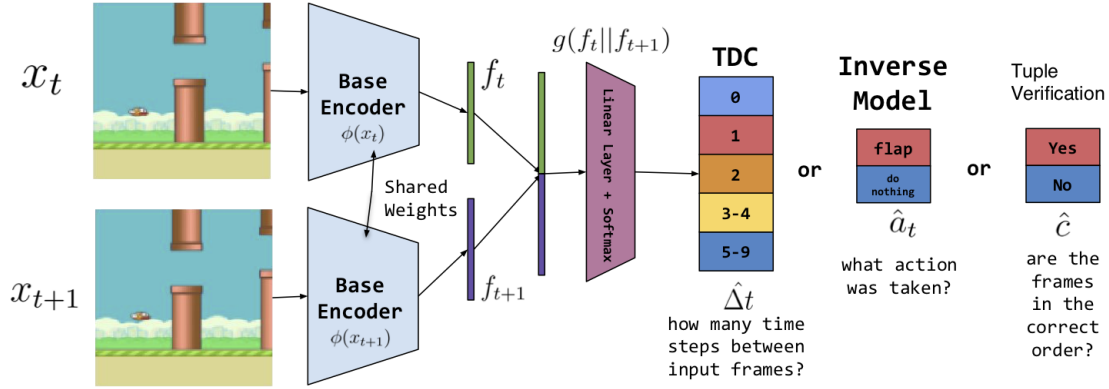


Figure 1. **General architecture for self-supervised embedding.** Shown for Flappy Bird. Two or three frames are each input to the base encoder then the outputs from the encoder, $\phi(x)$ are concatenated and passed to a linear softmax layer that classifies either a) "how many time steps are between a pair of frames?" for the TDC model (Aytar et al., 2018), b) "what action was taken to go from the first frame to second?" for the inverse model (Agrawal et al., 2016), or c) "are a triplet of frames in the correct chronological order?" for the tuple verification model (Misra et al., 2016)

compelling feature map for each frame, which we show in figures 5 and 3. Confirming our hypothesis from 3.2, we see for Flappy Bird, the inverse model feature map focuses on the bird and the TDC feature map focuses on the pipe. Interestingly enough, tuple verification keys in on the top half of the pipe and the VAE activates on everything in the frame, but the pipes. For Sonic, things are not as clean and interpretable. As we see in figure 3, the inverse model feature map and the TDC one focus in on a cloud, perhaps mistaking it for Sonic, and the tuple verification map keys in on nothing at all. The VAE feature map, unsurprisingly, activates on important, ubiquitous objects for reconstructing the frame, like the tree and the bush. None of these representative feature maps key in on Sonic himself, which agrees with the poor quantitative classification accuracy results in table 1.

4. Related Work

This paper is not the first paper to quantitatively and qualitatively compare features from self-supervised methods in interactive environments. (Burda et al., 2018) compare the feature spaces learned from VAE's, Inverse Models, and Random CNN's and raw pixels (but not sequence verification methods) across a large variety of games. They even measure the generalization of these feature spaces to new, unseen environments. However, all their evaluations are in the context of how well an agent explores its environment with this feature space using extrinsic rewards and other measures of exploration. Additionally, (Shelhamer et al., 2016) studies VAE's, Inverse Models, and a sequence verification task in RL environments, but they evaluate these self-supervised methods as auxiliary tasks paired with a traditional extrinsic reward policy gradient algorithm, A3C

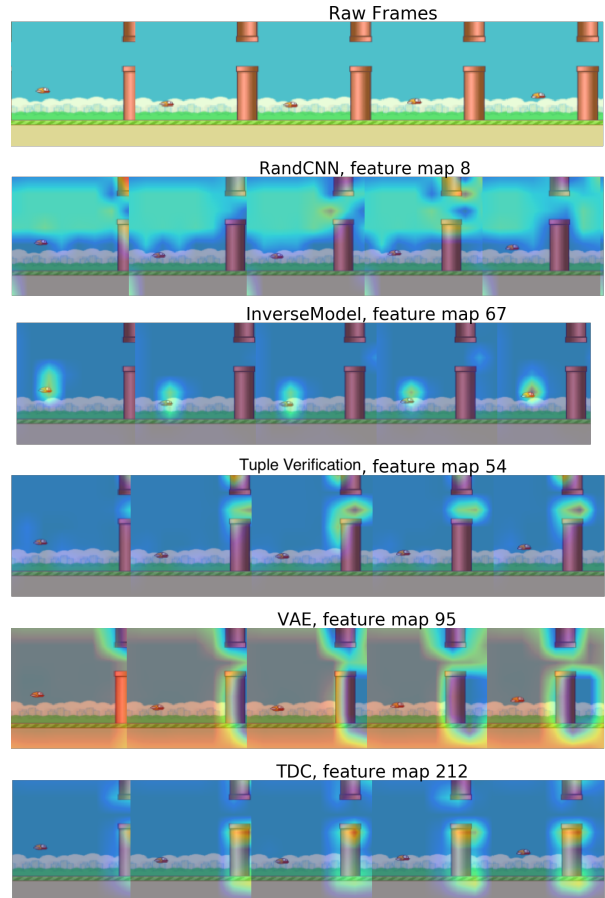


Figure 2. **Qualitative Inspection of Feature Maps** Flappy Bird feature maps from the last conv layer of the encoder superimposed on top of a sequence of frames they are a function of. Red pixels are high values, blue are low values

Table 1. Extracting Position Information We train a linear classifier on top of feature spaces from each method and measure the classification accuracy for the various position of various objects in the game, y position of the bird and x position of the first pipe for Flappy Bird and y position of Sonic in Sonic

METHOD	FLAPPY BIRD		SONIC
	BIRD Y Pos ACC (%)	PIPE X Pos ACC(%)	Y Pos (%)
RANDCNN	35.57	52.13	23.88
VAE	36.20	76.52	53.82
INV MODEL	91.67	81.64	25.83
TUPLE VERIFICATION	75.41	87.08	11.2
TDC	56.06	92.36	10.42

Table 2. Generalizing Extracting Position to New Levels and Colors We see how well the trained linear classifiers do in a zero shot transfer to new colors for Flappy Bird and a new level for Sonic

METHOD	FLAPPYBIRDNIGHT		SONIC GREENHILLZONE ACT 2
	BIRD Y Pos ACC (%)	PIPE X Pos ACC(%)	Y Pos (%)
RANDCNN	34.03	7.41	28.31
VAE	2.53	2.0	34.57
INV MODEL	36.5	8.90	27.28
TUPLE VERIFICATION	1.88	1.74	14.4
TDC	46.9	16.22	13.59



Figure 3. Sonic feature maps from the last conv layer of the encoder superimposed on top of the frames they are a function of for from left: random CNN, VAE, inverse Model, tuple verification, and temporal distance classification. Red is high values, blue are low values

(Mnih et al., 2016), using empirical returns from extrinsic rewards as a measure of utility of each feature space. Lastly, trying to infer the position of salient objects has been explored a lot in robotics in a field called state inference (Jonschkowski & Brock, 2015; Jonschkowski et al., 2017). Moreover, (Raffin et al., 2018; Lesort et al., 2018) look into using some self-supervised tasks for state inference, but they mostly measure performance on control tasks; they do not measure direct correlation or classification accuracy of the features to the true position of the object.

5. Discussion

We have shown comparing methods on interactive environments reveals intriguing things about the self-supervised methods as well as the environments themselves. Particularly, we expose various traits of environments that some self-supervised tasks can take advantage of and others cannot. For example, inverse models are very good at localizing what they can control even it is small, but only when the

dynamics are simple and predictable and the appearance of the agent itself is consistent. Temporal distance classifiers are very good at capturing things that move very predictably in time. Tuple verification encoders are good at capturing small and large objects in environments with pretty consistent graphics and dynamics. VAE’s learn good features when the objects are big and repeatably show up in the scene with consistent appearance.

5.1. Future Work

The very different behaviors of each method depending on the traits of the environment warrants further study covering more environments with more diverse appearances and dynamics, as well as a wider range of self-supervised methods. In addition, the fact that some methods excel at capturing or generalizing better than others depending on the environment motivates exploring potentially combining these methods by having a shared encoder body with multiple self-supervised heads. We hope that this study can open the

door to more extensive, rigorous approaches for studying the capability of self-supervised methods and that its results can inspire new methods that learn even better features.

References

- Agrawal, P., Carreira, J., and Malik, J. Learning to see by moving. In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 37–45, 2015.
- Agrawal, P., Nair, A. V., Abbeel, P., Malik, J., and Levine, S. Learning to poke by poking: Experiential learning of intuitive physics. In *Advances in Neural Information Processing Systems*, pp. 5074–5082, 2016.
- Anonymous. Exploration by random distillation. 2018. URL <https://openreview.net/pdf?id=H11JJnR5Ym>. Submitted to ICLR 2019.
- Arora, S., Khandeparkar, H., Khodak, M., Plevrakis, O., and Saunshi, N. A theoretical analysis of contrastive unsupervised representation learning. *arXiv preprint arXiv:1902.09229*, 2019.
- Aytar, Y., Pfaff, T., Budden, D., Paine, T. L., Wang, Z., and de Freitas, N. Playing hard exploration games by watching youtube. *arXiv preprint arXiv:1805.11592*, 2018.
- Burda, Y., Edwards, H., Pathak, D., Storkey, A., Darrell, T., and Efros, A. A. Large-scale study of curiosity-driven learning. *arXiv preprint arXiv:1808.04355*, 2018.
- Choi, J., Guo, Y., Moczulski, M., Oh, J., Wu, N., Norouzi, M., and Lee, H. Contingency-aware exploration in reinforcement learning. *arXiv preprint arXiv:1811.01483*, 2018.
- Conneau, A. and Kiela, D. Senteval: An evaluation toolkit for universal sentence representations. *arXiv preprint arXiv:1803.05449*, 2018.
- Fernando, B., Gavves, E., Oramas, J. M., Ghodrati, A., and Tuytelaars, T. Modeling video evolution for action recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 5378–5387, 2015.
- Fernando, B., Bilen, H., Gavves, E., and Gould, S. Self-supervised video representation learning with odd-one-out networks. In *Computer Vision and Pattern Recognition (CVPR), 2017 IEEE Conference on*, pp. 5729–5738. IEEE, 2017.
- Ha, D. and Schmidhuber, J. World models. *arXiv preprint arXiv:1803.10122*, 2018.
- Hyvarinen, A. and Morioka, H. Unsupervised feature extraction by time-contrastive learning and nonlinear ica. In *Advances in Neural Information Processing Systems*, pp. 3765–3773, 2016.
- Hyvarinen, A., Sasaki, H., and Turner, R. E. Nonlinear ica using auxiliary variables and generalized contrastive learning. *arXiv preprint arXiv:1805.08651*, 2018.
- Jaderberg, M., Mnih, V., Czarnecki, W. M., Schaul, T., Leibo, J. Z., Silver, D., and Kavukcuoglu, K. Reinforcement learning with unsupervised auxiliary tasks. *arXiv preprint arXiv:1611.05397*, 2016.
- Jayaraman, D. and Grauman, K. Learning image representations tied to ego-motion. In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 1413–1421, 2015.
- Jonschkowski, R. and Brock, O. Learning state representations with robotic priors. *Autonomous Robots*, 39(3): 407–428, 2015.
- Jonschkowski, R., Hafner, R., Scholz, J., and Riedmiller, M. Pves: Position-velocity encoders for unsupervised learning of structured state representations. *arXiv preprint arXiv:1705.09805*, 2017.
- Kingma, D. P. and Welling, M. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- LeCun, Y. Learning world models with self-supervised learning, 2018. Presented at ICML workshop on Generative Modeling in RL.
- Lesort, T., Díaz-Rodríguez, N., Goudou, J.-F., and Filliat, D. State representation learning for control: An overview. *Neural Networks*, 2018.
- Mikolov, T., Chen, K., Corrado, G., and Dean, J. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.
- Mirowski, P., Pascanu, R., Viola, F., Soyer, H., Ballard, A. J., Banino, A., Denil, M., Goroshin, R., Sifre, L., Kavukcuoglu, K., et al. Learning to navigate in complex environments. *arXiv preprint arXiv:1611.03673*, 2016.
- Misra, I., Zitnick, C. L., and Hebert, M. Shuffle and learn: unsupervised learning using temporal order verification. In *European Conference on Computer Vision*, pp. 527–544. Springer, 2016.
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., and Kavukcuoglu, K. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, pp. 1928–1937, 2016.

- Nichol, A., Pfau, V., Hesse, C., Klimov, O., and Schulman, J. Gotta learn fast: A new benchmark for generalization in rl. *arXiv preprint arXiv:1804.03720*, 2018.
- Pathak, D., Agrawal, P., Efros, A. A., and Darrell, T. Curiosity-driven exploration by self-supervised prediction. 2017a.
- Pathak, D., Girshick, R., Dollár, P., Darrell, T., and Hariharan, B. Learning features by watching objects move. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6024–6033. IEEE, 2017b.
- Peters, M. E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K., and Zettlemoyer, L. Deep contextualized word representations. *arXiv preprint arXiv:1802.05365*, 2018.
- Raffin, A., Hill, A., Traoré, R., Lesort, T., Díaz-Rodríguez, N., and Filliat, D. S-rl toolbox: Environments, datasets and evaluation metrics for state representation learning. *arXiv preprint arXiv:1809.09369*, 2018.
- Sermanet, P., Lynch, C., Chebotar, Y., Hsu, J., Jang, E., Schaal, S., and Levine, S. Time-contrastive networks: Self-supervised learning from video. *arXiv preprint arXiv:1704.06888*, 2017.
- Shelhamer, E., Mahmoudieh, P., Argus, M., and Darrell, T. Loss is its own reward: Self-supervision for reinforcement learning. *arXiv preprint arXiv:1612.07307*, 2016.
- Subramanian, S., Trischler, A., Bengio, Y., and Pal, C. J. Learning general purpose distributed sentence representations via large scale multi-task learning. *arXiv preprint arXiv:1804.00079*, 2018.
- Tasfi, N. Pygame learning environment. <https://github.com/ntasfi/PyGame-Learning-Environment>, 2016.
- Vondrick, C., Shrivastava, A., Fathi, A., Guadarrama, S., and Murphy, K. Tracking emerges by colorizing videos. In *European Conference on Computer Vision*, pp. 402–419. Springer, 2018.
- Wang, X. and Gupta, A. Unsupervised learning of visual representations using videos. In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 2794–2802, 2015.
- Wei, D., Lim, J. J., Zisserman, A., and Freeman, W. T. Learning and using the arrow of time. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 8052–8060, 2018.

Appendix

A. More Details on The Self-Supervised Methods We Explore

Code available at <https://github.com/eracah/self-supervised-survey> Most of the self-supervised tasks work like so:

$$\hat{o} = g([f_i, \dots, f_j]) \quad (1)$$

where $[f_i, \dots, f_j]$ is a concatenation of anywhere from 1 to 3 embeddings depending on the self-supervised method, where g is a deconvolutional decoder for VAE’s and a linear classifier for the other self-supervised methods and:

$$f_t, f_{t+1} = \phi(x_t), \phi(x_{t+1}) \quad (2)$$

The meaning of $\hat{o}$ varies depending on the task **Random CNN**: We use a randomly initialized CNN as a baseline method. In this case, it is an untrained base encoder, ϕ , with randomly initialized weights that are not updated. Random CNN’s have been used with varying degrees of success in (Burda et al., 2018; Anonymous, 2018)

VAE: VAE’s (Kingma & Welling, 2013) are latent variable models that maximize a lower bound of the data likelihood, $p(x)$ by approximating the posterior $p(z|x)$, with a parametric distribution $q(z|x)$ and a prior $p(z)$. VAE’s also include a decoder $p(x|z)$, which reconstructs the input x by mapping samples of z back to pixel space. In our setup $q(z|x)$ is parametrized by a gaussian with mean $\phi(x)$ and the variance is parametrized by a separate fully connected layer on top of the penultimate layer of the base encoder. Also, we use a deconvolutional network, $\hat{x} = g(z)$ to parametrize $p(x|z)$. The VAE is trained by minimizing the KL divergence between the prior, $p(z)$, which we often pick to be an isotropic gaussian, and the approximate posterior, $q(z|x)$, while also minimizing the negative log-likelihood of $p(x|z)$, like so:

$$KL(q(z|x)||p(z)) - \log p(x|z) \quad (3)$$

The idea is that if we learn close to factorized latent variables that encode enough information to reliably reconstruct the frame, they will capture important structure of the image, like objects.

Temporal Distance Classification (TDC): Temporal Distance Classification (TDC) is a self-supervised method introduced by (Aytar et al., 2018), similar to (Hyvarinen & Morioka, 2016), where the network learns the relative distance in time between frames. In TDC, we do the following:

$$\hat{\Delta}t = g([f_t, f_{t+\Delta t}]) \quad (4)$$

$[f_t, f_{t+\Delta t}]$ is the concatenation of the embeddings of frames x_t and $x_{t+\Delta t}$, and where Δt is sampled from a set of intervals, $\Delta t \in D$

$$D = \{[0], [1], [2], [3-4], [5-9]\} \quad (5)$$

where g is a k -way linear classifier, where $k = |D|$ that classifies which time interval separates the two input frames. The idea is that in order to do well at the task, it must learn the features of the input that change over time, which often corresponds to objects (Aytar et al., 2018).

Tuple Verification: Tuple Verification (Misra et al., 2016) is an instance of a temporal order or dynamics verification task, where the network must figure out if a sequence of frames is in order or not. The method works as such: five chronologically ordered, evenly-spaced (they don't have to be consecutive) frames are chosen from a rollout:

$$\{x_a, x_b, x_c, x_d, x_e : a < b < c < d < e\} \quad (6)$$

In this paper we use an evenly spaced sampling of 5 frames from a sequence of 10 consecutive frames. A binary classification problem is created by shuffling the frames to create negative examples. A tuple of frames in the order x_b, x_c, x_d is a positive example, whereas the tuples (x_b, x_e, x_d) and (x_b, x_a, x_d) are negative examples. We ensure that there is a 2 : 1 : 1 ratio between positive samples and the two types of negative samples. The three frames are each encoded with the base encoder, concatenated and passed to a linear classifier:

$$\hat{c} = g([f_b, f_c, f_d]), \hat{c} = g([f_b, f_e, f_d]), \hat{c} = g([f_b, f_a, f_d]) \quad (7)$$

where g is linear binary classifier the predicts whether the frames are in order or not. Being successful at this task requires knowing how objects transform and move over time, which requires encoding of features corresponding to the appearance and location of objects (Misra et al., 2016).

Inverse Model: The inverse model (Agrawal et al., 2016; Jayaraman & Grauman, 2015; Pathak et al., 2017a) works by taking two consecutive frames from a rollout from an agent, then classifying which action was taken to transition from one frame to other. The model works like so:

$$\hat{a}_{t+1} = g([f_t, f_{t+1}]) \quad (8)$$

where g is a k -way linear classifier, where k is the size of the action space, the number of possible actions the agent can take. The idea is that in order to reason about which action was taken, the network must learn to focus on parts of the environment are controllable and affect the agent (Choi et al., 2018). This should then result in the network learning features that capture the agent's state and location as well potential obstacles to the agent.

B. More on Dataset Collection

We edit the action space of Sonic to be just the two actions: ["Right"] and ["Down", "B"]. This ensures the random agent will get pretty far in the level and actually collect a good diversity of frames. For both games, we resize the frames to 128 x 128. All ground truth position information (y position of bird, x position of pipe, y position of Sonic) is pulled from the gym API of these games and discretized to 16 buckets and represents the relative position of these objects in the frame, not the absolute position in the game. We purposely do not choose the x position of the bird or Sonic because in most frames of the game, the x position is relatively constant, while the background moves.

C. Prediction in Feature Space

To measure how predictive features are we train a one-step forward linear model, ψ that takes in the features at time step t , $f_t = \phi(x_t)$ and the action, a_t , and predicts the next step's features, $\hat{f}_{t+1}$, where the true future features are $f_{t+1} = \phi(x_{t+1})$ as seen in figure 4.

$$\hat{f}_{t+1} = \psi(f_t, a_t) \quad (9)$$

The loss is then the mean squared error between the true and predicted features:

$$L_f = \frac{1}{m} ||f_{t+1} - \hat{f}_{t+1}||_2^2 \quad (10)$$

where m is number of features in the embeddings, which in our case is 32.

Then at test time, we use the forward model to iteratively get a rollout of ten time steps worth of features:

$$\hat{f}_{t+1} = \psi(f_t, a_t), \quad (11)$$

$$\hat{f}_{t+2} = \psi(\hat{f}_{t+1}, a_{t+1}), \quad (12)$$

$$\hat{f}_{t+3} = \psi(\hat{f}_{t+2}, a_{t+2}), \quad (13)$$

$$\dots \quad (14)$$

$$\hat{f}_{t+9} = \psi(\hat{f}_{t+8}, a_{t+8}) \quad (15)$$

We then take the first principal component of each predicted feature and we look at Spearman correlation of the principal components with the true ground truth state information.

C.1. Results

We take a closer look at how "predictive" the features for each method are in table 3. Surprisingly, the predicted

features better capture this state information than the true features for many self-supervised tasks. This might be because prediction is easier for simple objects than it is for other (potentially more trivial) factors of variation, so the task of prediction slowly changes the feature space to capture less other factors of variation and more the ones for objects. In addition, in learning a dynamic task like prediction the forward model learns about the things that move. This could also be due to what (Agrawal et al., 2016) refers to as regularizing the feature space with a forward model. It is worth noting that the predicted inverse model features are very predictive, as they retain their stellar ability to capture the true agent position.

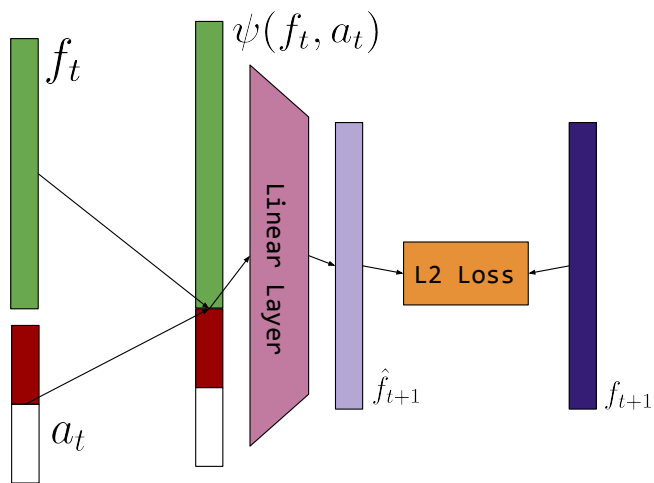


Figure 4. Predicting in Feature Space: Architecture for predicting in feature space: an embedding at time step is concatenated with the action at time t and put through a linear layer to get the predicted embedding at time step $t+1$

Table 3. Evaluating Prediction in Feature Space . We take the first principal component of the predicted feature vectors and compute the Spearman’s rank correlation coefficient with the true values: y position of the bird and x position of the first pipe for Flappy Bird and y position of the Sonic character in Sonic

METHOD	FLAPPY BIRD				SONIC	
	BIRD Y POS		PIPE X POS		SONIC Y POS	
	TRUE	PREDICTED	TRUE	PREDICTED	TRUE	PREDICTED
RANDCNN	-0.01	0.066	-0.078	0.0035	0.66	0.0044
VAE	-0.17	-0.21	-0.32	-0.59	0.52	0.011
INVMODEL	0.88	0.87	-0.092	0.04	0.04	-0.0059
SNL	0.0	0.23	-0.82	0.0027	0.0016	0.0063
TDC	-0.23	-0.10	0.34	-0.71	-0.21	0.0072

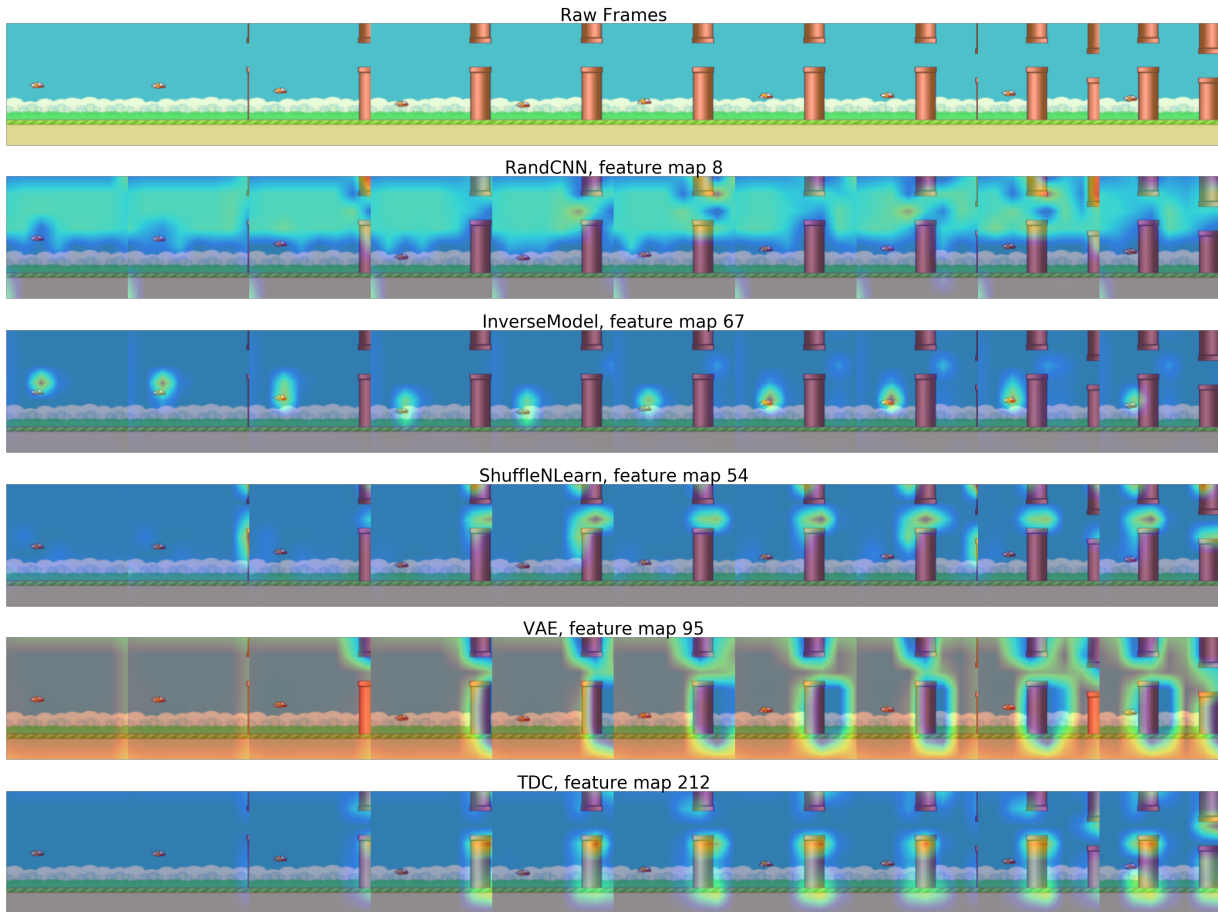


Figure 5. Qualitative Inspection of Feature Maps (longer version) Flappy Bird feature maps from the last conv layer of the encoder superimposed on top of a sequence of frames they are a function of. Red pixels are high values, blue are low values